

In []: Для выполнения заданий были выбраны следующие датасеты, содержащие реальные записи

1. Для классификации - MAGIC Gamma Telescope. Используется в задачах идентификации. Важно быстро и точно отличать полезный сигнал (гамма-кванты) от фоновых событий. Практическая ценность заключается в улучшении точности классификации, влияющей на...
2. Для регрессии - Air Quality. Используется в задачах прогнозирования концентрации загрязняющих веществ. Данные содержат почасовые записи результатов измерений, проводившихся в течение...

Оба датасета представляют собой результаты реальных измерений. Содержат шумы, предназначены для демонстрации навыков построения моделей и работы с ними, а также демонстрируют результаты научных исследований и повседневной жизни.

Для оценки качества моделей выбраны следующие метрики. Для классификации:

- Accuracy - показывает, какую долю объектов модель классифицировала правильно. В этом случае эта метрика даст адекватную оценку.
- F1 - учитывает оба класса и их ошибки, показывает точность (сколько из отнесённых объектов действительно принадлежат к классу) и полноту (сколько найдено настоящих объектов класса) классификации. В случае, если модель имеет высокий Accuracy, F1 покажет это.

Для регрессии:

- MAE (Mean Absolute Error) - средняя абсолютная ошибка. Показывает среднюю величину ошибки, не штрафует за большие ошибки сильнее, чем за маленькие.
- RMSE (Root Mean Squared Error) - корень из средней квадратичной ошибки. Показывает среднюю величину ошибки, сильнее штрафует за большие отклонения. Позволяет обнаружить наличие выбросов.
- R2 (коэффициент детерминации) - показывает долю дисперсии целевой переменной, которую модель объясняет. Чем ближе к 1, тем лучше. Если модель не лучше простого предсказания среднего значения, R2 будет отрицательным.

In []: *#1.1 KNN-классификация*

In []: Импорт необходимых библиотек:

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

In []: *## 1. Загрузка и обработка данных.*

Загружаем датасет в объект DataFrame (pandas). После настройки параметров выводим информацию о датасете и также для проверки корректности загрузки. Получаем общую информацию о датасете (индексный столбец Unnamed: 0), целевая переменная class типа object (строковый) 'h' адроны. Доли классов: 65% (12332) g, 35% (6688) h. Это говорит об умеренном

```
In [3]: df = pd.read_csv('C:/Users/Elizaveta/Downloads/magic_gamma/telescope_data.csv')
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
df.head(5)
df.info()
print("размер", df.shape)
print("названия столбцов", list(df.columns))
print("уникальные значения в class", df['class'].unique())
print("\nраспределение классов")
print(df['class'].value_counts())
print("доли классов")
print(df['class'].value_counts(normalize=True))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19020 entries, 0 to 19019
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0   19020 non-null  int64
1   fLength      19020 non-null  float64
2   fWidth       19020 non-null  float64
3   fSize        19020 non-null  float64
4   fConc        19020 non-null  float64
5   fConc1       19020 non-null  float64
6   fAsym        19020 non-null  float64
7   fM3Long      19020 non-null  float64
8   fM3Trans     19020 non-null  float64
9   fAlpha       19020 non-null  float64
10  fDist        19020 non-null  float64
11  class        19020 non-null  object
dtypes: float64(10), int64(1), object(1)
memory usage: 1.7+ MB
размер (19020, 12)
названия столбцов ['Unnamed: 0', 'fLength', 'fWidth', 'fSize', 'fConc', 'fConc1',
'fAsym', 'fM3Long', 'fM3Trans', 'fAlpha', 'fDist', 'class']
уникальные значения в class ['g' 'h']

распределение классов
class
g    12332
h     6688
Name: count, dtype: int64
доли классов
class
g    0.64837
h    0.35163
Name: proportion, dtype: float64
```

In []: Удаляем индексный столбец, не несущий полезной информации:

In [4]: `df=df.drop('Unnamed: 0', axis=1)`

In []: После удаления лишнего столбца требуется убедиться, что нет посторонних символов
Вывод первых десяти значений столбца `class` - убедились, что данные действительно
С помощью `repr()` проверяем наличие скрытых символов (пробелы и т.д.) - отсутству

In [5]: `print("first 10 values", df['class'].head(10))`
`print("unique values", df['class'].unique())`
`print("value counts", df['class'].value_counts())`
`print("type of data in class", df['class'].dtype)`
`print("first value", repr(df['class'].iloc[0]))`

```

first 10 values 0      g
1      g
2      g
3      g
4      g
5      g
6      g
7      g
8      g
9      g
Name: class, dtype: object
unique values ['g' 'h']
value counts class
g      12332
h      6688
Name: count, dtype: int64
type of data in class object
first value 'g'

```

In []: Преобразуем целевую переменную в числовой формат, после того, как убедились в чи
преобразования. Каждое вхождение 'g' заменяется на 1, 'h' - на 0. Все данные успе

```

In [6]: df['class']=df['class'].replace({'g': 1, 'h': 0})
print("first 10 values", df['class'].head(10))
print("unique values", df['class'].unique())
print("value counts", df['class'].value_counts())
print("type of data in class", df['class'].dtype)
print("first value", repr(df['class'].iloc[0]))
print("NaN", df['class'].isna().sum())

```

```

first 10 values 0      1
1      1
2      1
3      1
4      1
5      1
6      1
7      1
8      1
9      1
Name: class, dtype: int64
unique values [1 0]
value counts class
1      12332
0      6688
Name: count, dtype: int64
type of data in class int64
first value np.int64(1)
NaN 0

```

C:\Users\Elizaveta\AppData\Local\Temp\ipykernel_17176\1776076825.py:1: FutureWarn
ing: Downcasting behavior in `replace` is deprecated and will be removed in a fut
ure version. To retain the old behavior, explicitly call `result.infer_objects(co
py=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silen
t_downcasting', True)`
df['class']=df['class'].replace({'g': 1, 'h': 0})

In []: Разделение данных на признаки X и целевую переменную Y. После чего проверяем раз

```

In [7]: X=df.drop('class', axis=1)
Y=df['class']

```

```
print("size of X", {X.shape})
print("size of Y", {Y.shape})
```

```
size of X {(19020, 10)}
size of Y {(19020,)} 
```

In []: Разделение данных на обучающую (train) и тестовую (test) выборки. Обучаем модель участвовавшей в обучении. Функция train_test_split разделяет данные случайным образом random_state=42 стандартное значение, начальное состояние генератора случайных чисел - стратификация по целевой переменной = пропорции классов в обучающей и тестовой несбалансированных выборках). Выводим размер полученных выборок и баланс классов

```
In [8]: X_train, X_test, Y_train, Y_test = train_test_split(
        X, Y,
        test_size=0.3,
        random_state = 42,
        stratify = Y)
print(f"size of X_train", {X_train.shape})
print(f"size of X_test", {X_test.shape})
print(f"size of Y_train", {Y_train.shape})
print(f"size of Y_test", {Y_test.shape})
print("balance of classes:")
print(f"start data: 1={Y.mean():.3f}, 0={1-Y.mean():.3f}")
print(f"train data: 1={Y_train.mean():.3f}, 0={1-Y_train.mean():.3f}")
print(f"start data: 1={Y_test.mean():.3f}, 0={1-Y_test.mean():.3f}")
```

```
size of X_train {(13314, 10)}
size of X_test {(5706, 10)}
size of Y_train {(13314,)}
size of Y_test {(5706,)}
balance of classes:
start data: 1=0.648, 0=0.352
train data: 1=0.648, 0=0.352
start data: 1=0.648, 0=0.352
```

In [9]: *## 2. Создание бейзлайна и оценка качества.*
Обучаем модель KNeighborsClassifier из sklearn с параметрами по умолчанию (n_neighbors=5). F1 учитывает оба класса и их ошибки, показывает точность (сколько из отнесённых и полноту (сколько найдено настоящих объектов класса) классификации. В случае, если при высоком Accurasy, F1 покажет это. Accurasy показывает, какую долю объектов модель классифицировала правильно. В данном случае, поэтому эта метрика даст адекватную оценку. Обучающая выборка (train) используется для обучения модели. Тестовая выборка (test) используется для оценки качества обученной модели. Результат на бейзлайне: accuracy_test=0.86, accuracy_train=0.81, f1_train=0.86, f1_test=0.81. Получены достаточно хорошие результаты, но есть потенциал для улучшения. Эти данные можно использовать для улучшения, показывающей, чего можно достичь минимальными

Cell In[9], line 2

Обучаем модель KNeighborsClassifier из sklearn с параметрами по умолчанию (n_neighbors=5). Использованные метрики - F1-score и Accurasy.

^
SyntaxError: invalid syntax

```
In [30]: from sklearn.neighbors import KNeighborsClassifier
        from sklearn.metrics import accuracy_score, f1_score
        knn_baseline = KNeighborsClassifier(n_neighbors=5)
        knn_baseline.fit(X_train, Y_train)
        Y_train_prediction = knn_baseline.predict(X_train)
        Y_test_prediction = knn_baseline.predict(X_test)
        accuracy_train = accuracy_score(Y_train, Y_train_prediction)
```

```

accuracy_test = accuracy_score(Y_test, Y_test_prediction)
f1_train = f1_score(Y_train, Y_train_prediction)
f1_test = f1_score(Y_test, Y_test_prediction)
print(f'accuracy train: {accuracy_train:.2f}')
print(f'accuracy test: {accuracy_test:.2f}')
print(f'f1 train: {f1_train:.2f}')
print(f'f1 test: {f1_test:.2f}')

```

```

accuracy train: 0.86
accuracy test: 0.81
f1 train: 0.90
f1 test: 0.86

```

In []: *## 3. Улучшение бейзлайна.*
Гипотеза 1: Масштабирование признаков.
 KNN использует расстояния между точками, поэтому признаки, имеющие разные единицы
 Стандартизация может улучшить качество классификации.
 Применяем StandardScaler к признакам (обучение на train, преобразование на train)

```

In [13]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
knn_scaled = KNeighborsClassifier(n_neighbors=5)
knn_scaled.fit(X_train_scaled, Y_train)
Y_train_scaled_prediction = knn_scaled.predict(X_train_scaled)
Y_test_scaled_prediction = knn_scaled.predict(X_test_scaled)
accuracy_train = accuracy_score(Y_train, Y_train_scaled_prediction)
accuracy_test = accuracy_score(Y_test, Y_test_scaled_prediction)
f1_train = accuracy_score(Y_train, Y_train_prediction)
f1_test = accuracy_score(Y_test, Y_test_prediction)
print(f'accuracy train: {accuracy_train:.2f}')
print(f'accuracy test: {accuracy_test:.2f}')
print(f'f1 train: {f1_train:.2f}')
print(f'f1 test: {f1_test:.2f}')

```

```

accuracy train: 0.88
accuracy test: 0.83
f1 train: 0.86
f1 test: 0.81

```

In [17]: *### Гипотеза 2: Подбор гиперпараметров.*
 Значение n_neighbors по умолчанию может быть неоптимальным. Также проверим влияни
 Оптимальные параметры находим с помощью кросс-валидации.
 Используем GridSearchCV с 5-кратной кросс-валидацией на масштабированных данных.
 В результате найдены значения: n_neighbors=10, manhattan, uniform. Результат луч
 по сравнению с масштабированием без подбора - улучшение на 1.02%.
 Таким образом, обе гипотезы подтвердились и эти улучшения подходят для использо

Cell In[17], line 2

Значение n_neighbors по умолчанию может быть неоптимальным. Также проверим вл
 ияние весов и метрики расстояния.

^

SyntaxError: invalid syntax

```

In [27]: from sklearn.model_selection import GridSearchCV
param_grid = {'n_neighbors': range(1,31),
              'weights': ['uniform', 'distance'],
              'metric': ['euclidean', 'manhattan']}
knn_grid = KNeighborsClassifier()
grid_search = GridSearchCV(knn_grid, param_grid, cv=5, scoring='accuracy')

```

```

grid_search.fit(X_train_scaled, Y_train)
best_params = grid_search.best_params_
best_score = grid_search.best_score_
print(best_params)
print(best_score)
best_knn = grid_search.best_estimator_
Y_test_prediction_best = best_knn.predict(X_test_scaled)
accuracy_best = accuracy_score(Y_test, Y_test_prediction_best)
print(f'accuracy best: {accuracy_best:.4f}')
print(f'baseline (not scaled, n=5): 0.81")

```

```

{'metric': 'manhattan', 'n_neighbors': 10, 'weights': 'uniform'}
0.843247999056554
accuracy best: 0.8402
baseline (not scaled, n=5): 0.81

```

In []: *## 4. Имплементация алгоритма машинного обучения.*
 Создаем свой классификатор MyKNNclassifier со стандартными параметрами. Основные
 __init__ - задание гиперпараметров (здесь используем стандартные).
 fit - сохранение обучающих данных.
 _compute_distance - вычисление расстояний.
 predict - формирование предсказания.
 MyKNNclassifier показал accuracy=0.8062 на тестовой выборке, практически совпада

```

In [28]: import numpy as np
from collections import Counter

class MyKNN:
    def __init__(self, n_neighbors=5, weights='uniform', metric='euclidean'):
        self.n_neighbors = n_neighbors
        self.weights = weights
        self.metric = metric
        self.X_train = None
        self.Y_train = None
    def fit(self, X, Y):
        self.X_train = np.array(X)
        self.Y_train = np.array(Y)
    def check(self):
        if self.X_train is None or self.Y_train is None:
            print("модель не обучена")
            return
        if len(self.X_train.shape)>1:
            num_features = self.X_train.shape[1]
        else:
            num_features = 1
        print(f"кол-во обучающих примеров {len(self.X_train)}")
        print(f"кол-во признаков {num_features}")
        print(f"кол-во меток классов {len(self.Y_train)}")
        print(f"уникальные классы {np.unique(self.Y_train)}")
    def _compute_distance(self, point1, point2):
        if self.metric == 'euclidean':
            return np.sqrt(np.sum((point1-point2)**2))
        elif self.metric == 'manhattan':
            return np.sum(np.abs(point1-point2))
        else:
            raise ValueError(f"Метрика {self.metric} не поддерживается")
    def _get_neighbors(self, point):
        distances = []
        for i, train_point in enumerate(self.X_train):
            distance = self._compute_distance(point, train_point)

```

```

        distances.append((i, distance))
    distances.sort(key=lambda x: x[1])
    neighbors = distances[:self.n_neighbors]
    return neighbors
def predict(self, X):
    if self.X_train is None or self.Y_train is None:
        raise ValueError("Модель не обучена")
    X = np.array(X)
    predictions = []
    for point in X:
        neighbors = self._get_neighbors(point)
        neighbor_classes = [self.Y_train[idx] for idx, _ in neighbors]
        if self.weights == 'uniform':
            class_counts = Counter(neighbor_classes)
            predicted_class = class_counts.most_common(1)[0][0]
        elif self.weights == 'distance':
            class_counts = Counter(neighbor_classes)
            predict_class = class_counts.most_common(1)[0][0]
        else:
            raise ValueError(f"weights='{self.weights}' не поддерживается")
        predictions.append(predicted_class)
    return np.array(predictions)

```

In [29]: Ниже представлены результаты работы собственного классификатора. Модель обучается. Получены результаты accuracy: 0.8062, f1: 0.8611, практически идентично бейзлай

Cell In[29], line 1

Ниже представлены результаты работы собственного классификатора:

^

SyntaxError: invalid syntax

```

In [31]: from sklearn.metrics import accuracy_score, f1_score
my_knn = MyKNN(n_neighbors=5, weights='uniform', metric='euclidean')
my_knn.fit(X_train, Y_train)
Y_pred_my_knn = my_knn.predict(X_test)
accuracy_my_knn = accuracy_score(Y_test, Y_pred_my_knn)
f1_my_knn = f1_score(Y_test, Y_pred_my_knn)
print(f"MyKNN accuracy: {accuracy_my_knn:.4f}")
print(f"MyKNN f1: {f1_my_knn:.4f}")

```

MyKNN accuracy: 0.8062

MyKNN f1: 0.8611

In []: Используем на собственном классификаторе подобранные оптимальные параметры и масштаб. accuracy улучшилась на 4,1% (с 0.8062 до 0.8395), f1 на 2,9% (с 0.8611 до 0.8862)

```

In [32]: my_knn_improved = MyKNN(n_neighbors=10, weights='uniform', metric='manhattan')
my_knn_improved.fit(X_train_scaled, Y_train)
Y_pred_my_knn_improved = my_knn_improved.predict(X_test_scaled)
accuracy_my_knn_improved = accuracy_score(Y_test, Y_pred_my_knn_improved)
f1_my_knn_improved = f1_score(Y_test, Y_pred_my_knn_improved)
print(f"MyKNN improved accuracy: {accuracy_my_knn_improved:.4f}")
print(f"MyKNN f1: {f1_my_knn_improved:.4f}")

```

MyKNN improved accuracy: 0.8395

MyKNN f1: 0.8862

In []: ## Выводы:

В ходе выполнения задания был выбран и загружен датасет, создана базовая модель. Результаты accuracy=0.81, f1=0.86, что говорит о хорошем распознавании. После ма

После подбора гиперпараметров также были выявлены улучшения, результат лучшей модели по сравнению с масштабированием без подбора - улучшение на 1.02%. Был реализован практически идентичный базовой модели, accuracy: 0.8062, f1: 0.8611, что говорит об оптимальности параметров, результаты повысились до accuracy: 0.8395 (на 4.1%) и f1: 0.8611, что говорит о том, что модель хорошо справляется с классификацией.